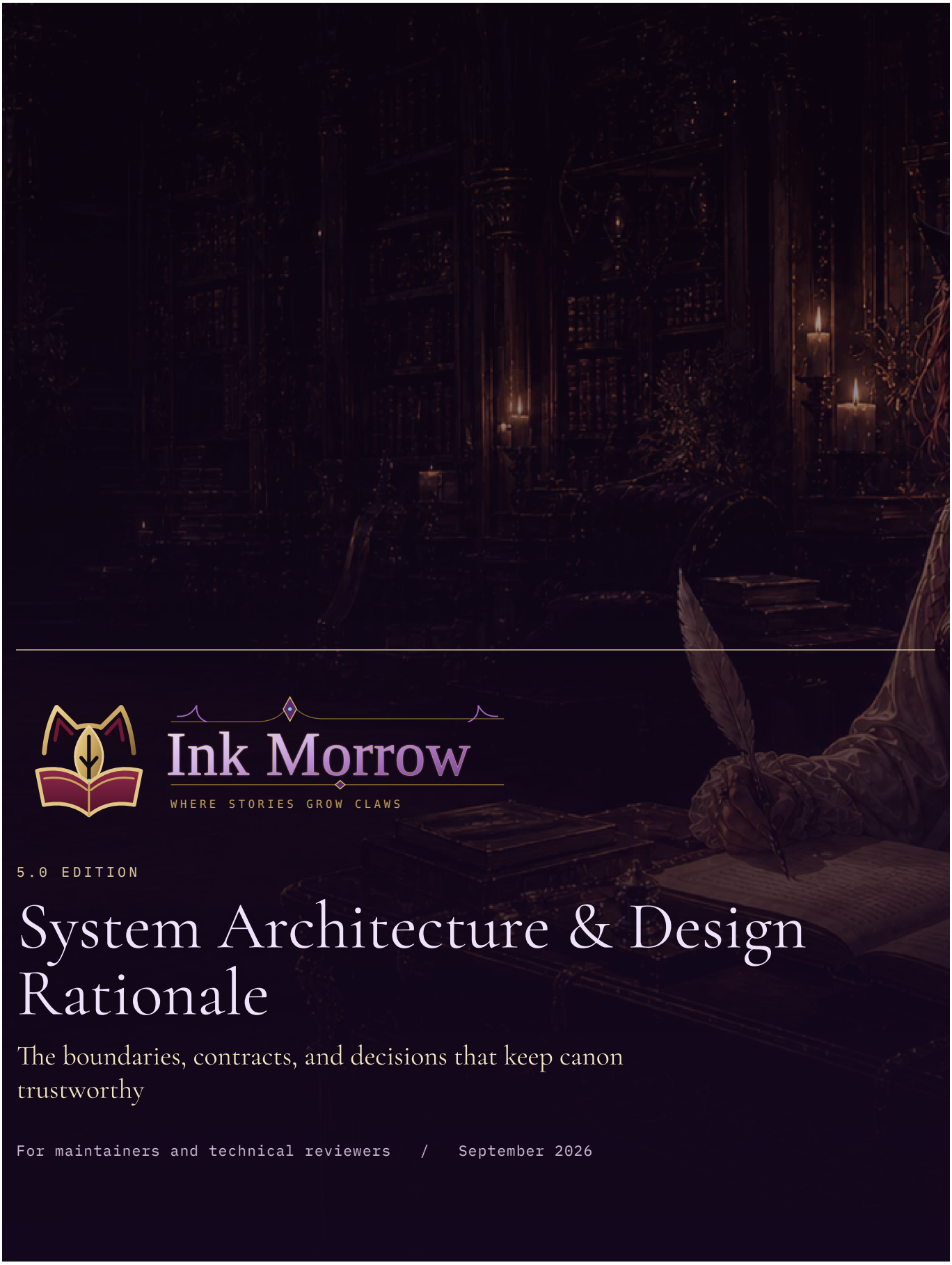


Ink Morrow

WHERE STORIES GROW CLAWS

5.0 EDITION

System Architecture & Design Rationale

The boundaries, contracts, and decisions that keep canon trustworthy

For maintainers and technical reviewers / September 2026

System Architecture & Design Rationale

InkMorrow 5.0 is a single-owner playable-fiction system. This book explains the current production boundaries, reuse decisions and limits; historical writing-suite routes are not part of its runtime.

Contents

Product and system context
Module and responsibility map
Fresh storage and startup order
The playable graph
Canon commit and spend accounting
Reader, privacy and asynchronous UI
Limits, performance and extension decisions
Bounded consistency pipeline
People and episode framing
Fourth-wall permission
Clear influence and evidence
Memory and adjudication foundations
Media and portability boundary

01 Product and system context

The durable product is one readable story with alternate paths, not a transcript that must later be converted into canon. The normal player is a reader-director outside the cast. Follow and Steer are complete experiences; Inhabit grants explicit control of one cast member and can be released.

An Express process serves static native-JavaScript modules and a protected API. SQLite holds identity, auth, providers and playable state. Normalized raster files live under a configured private media root. Remote models are untrusted proposal generators reached only at reviewed purchase boundaries.

The production composer creates auth, provider and fiction services. It does not instantiate the retired writing, continuity, catalogue, image-backfill, audiobook, transfer or public-share runtime. legacy-runtime.js is an explicit internal test and reuse seam; server.js always selects legacyEnabled:false.

Historical tables and migrations remain for tested infrastructure reuse. Their presence is not a compatibility promise or a live product surface. The database family is new, and old databases are refused before normal SQLite startup.

No per-character background agent, speculative continuation or offline simulation runs. A player's return does not advance fictional time, and local recaps do not call a provider.

The B1 current-fiction workspace uses `workspace-state.js` for normalized room/task routes and bounded device preferences/bookmarks, and `workspace-shell.js` for the five room adapters and shared contribution sheet. The existing app composes domain actions and preserves one story authority. Same-story room changes keep the lifecycle epoch and paid operation intact; story/global changes invalidate stale callbacks. No legacy runtime or second transcript is mounted.

Bookmarks contain only story/path/passage identities and a relative paragraph anchor, never prose or private facts. Restoration outside the loaded window uses existing ancestry-checked history/evidence reads and validates revision/path before painting. Invalid positions fall back with an explanation. Chronicle renders at most eighty loaded history records; Codex explicitly uses current public state and Gallery labels its incomplete loaded collection. Complete historical projections and unplaced-art queries are later batches.

02 Module and responsibility map

Component	Responsibility
server.js / core/storage.js	Environment, safe bind, shared data-path selection
app.js	Security order, service composition, current routers, static delivery
modules/auth and providers	Owner sessions, CSRF, credentials and logical roles
fiction/model.js	Strict domain shapes and deterministic transitions
fiction/store	Immutable history, ancestry, revisions and transactions
fiction/service.js and memory.js	Bounded relevant authoritative context
fiction/resistance.js and director.js	Structured rulings and scene opportunities
fiction/service and quality modules	Reviewed, bounded model work and accounting
fiction/media, publication, saves	Path-local art, reader-safe books, private copies
fiction/library and library-model	Reusable visual references, frozen setup copies, separate image journal
frontend/app/fiction	Shelf, start, reader, controls, dialogs and route fencing

Stores and domain validators should remain usable with isolated databases and mocked transport. UI components receive their dependencies rather than reimplementing authority checks. Rendering reader state is distinct from the private snapshot sent to a storyteller.

Common publication adapters and raster validation are reused without remounting the old authoring APIs. The canonical SVG lockup is also reused across README, app, authentication and manual rendering; typography is not independently recreated at each surface.

03 Fresh storage and startup order

The release identifies itself as 5.0.0, database family ink-morrow-5, schema 22 and SQLite application ID 0x494D3530. Package identities agree across root, backend, frontend and E2E. Playable saves separately identify their own format and version; these are not interchangeable schema numbers.

Before connecting SQLite to an existing source, preflight copies the database and its WAL/rollback journal into a unique private temporary directory. It does not copy shared memory; SQLite may rebuild that only beside the copy. Regular-file and source-change checks protect the inspection boundary.

Identity markers, supported version, migration ledger, quick_check and foreign keys must pass before the original enters normal WAL/migration/reconciliation. A recognized old family, unknown SQLite file, orphan journal, future version, invalid ledger or failed scratch copy is rejected. No fallback opens an old source merely because scratch space ran out.

The retained migration chain constructs the fresh schema transactionally. Migration rollback preserves the prior ledger/version. This does not make earlier product-family databases upgradeable. Operators must stop other writers; scratch preflight is not a live backup or an adversarial filesystem lock.

The shared storage resolver keeps server and terminal recovery on the same file. Default data is separate from the old series, and in-memory test runs receive disposable media storage rather than writing into the real installation.

04 The playable graph

A game identifies its active branch and optimistic revision. Each branch points to an exact historical head and optional parent/fork moment. Immutable beats link to their parent and carry the resulting state snapshot. Local corrections, preferences, control handoffs and episode actions are history too, distinct from narrated opening/scene prose and outside-story clarification.

A snapshot carries cast, knowledge, facts, commitments, relationships, resources, control, preferences, episode framing, director history, adjudications and image placements. Forking restores that exact state; switching branches does not merge their futures. A bounded reader window is not a lifetime history limit.

Every mutation checks the expected game revision and rejects concurrent changes. A pending paid operation excludes conflicting mutations. Network work runs outside the database transaction; the final commit rechecks ownership, revision, branch and reviewed provider plan before appending one accepted beat.

Initial facts and immutable changes are the authoritative memory history. A current 128-fact snapshot is only a working set. Retrieval chooses current versions on the selected ancestry; corrections supersede and retirement excludes, without deleting historical evidence. Sibling-path events cannot become context.

The graph is also the save boundary. Import validates all identities, references and ancestry before assigning fresh local IDs and committing a new game.

05 Canon commit and spend accounting

A reply begins with validated intent, expected revision and idempotency key. Successful replay returns the existing result without another provider call. Changed input under the same key is rejected. A repeated unchanged structured ruling is resolved locally before provider availability or quality checks.

For new work, the request is journalled, then each actual model call is recorded pending before dispatch. The proposal contains prose plus bounded structured effects. Local validation never makes an intermediate draft canonical. Quality, when enabled, can review and repair within its disclosed ceiling.

Commit appends the accepted beat, derived snapshot, request outcome and accounting in the same transaction after stale checks. Late or invalid output cannot append a scene. Its known charge still counts; uncertainty is represented explicitly rather than rounded to zero.

Per-call rows allow mixed known/unknown charges across one quality sequence. Parent-only historical entries are included only where no call rows exist, so the two accounting layers are not summed twice. Recent reader metadata is bounded and omits rejected prose and reviewer explanations.

Restart interrupts abandoned requests and calls. A late completion may improve the billing record but cannot revive the abandoned operation or bypass a changed story. Recovery reads are free and never invent a replacement purchase.

06 Reader, privacy and asynchronous UI

The static HTML begins gated. Authentication completes before private startup, story requests, provider requests or private rendering. Lock clears story text, cast, facts, drafts and credential fields; route/liveness tokens discard late responses that belong to an old story or locked session.

Controls show immediate loading or busy states before delayed network work. Reviewing and submitting are separate states, both protected against duplicate actions. Cancelling a review preserves the direction; a failed request reconciles through free reads without auto-submitting. Leaving a view does not claim to cancel transport already dispatched.

The reader receives filtered public records, not complete snapshots. Memory and evidence endpoints verify current ancestry and filter private or retired entries before result limits. Source links distinguish local records from narrated proof. Untrusted text is rendered as text, not executable HTML.

Provider context is a different boundary: relevant hidden facts and motives may be deliberately sent to the selected storyteller or reviewers. A model can leak them in prose despite instructions. Reader projection is mechanically filtered; semantic secrecy in generated text is not proven by that filtering.

Books consume the same reader-safe projection across formats. Saves intentionally contain private state and all paths, and therefore require a different exposure description.

07 Limits, performance and extension decisions

Bound	Purpose
24 cast members	Bounded active character context
128 facts in a snapshot	Bounded working set, not erased lifetime history
12 effects per response	Inspectable deterministic mutation
40 paths per story	Bounded alternate-history graph
80 shelf rows per page	Reach all stories without an unbounded response
32 recall results	Useful search without downloading all memory
200 placed moments / 400 retained assets	Bound current and historical art
10,000 moments in a save	Conservative portable-graph processing

Context uses bounded recent prose and relevant historical records rather than whole-manuscript replay. No embeddings server or local background model is required. Limits reject work before purchases where known; they do not justify deleting a player's commitments or pretending an unknown charge is zero.

Future extensions should preserve one authoritative graph, explicit paid authority and distinct reader/provider/save projections. A new optional model call needs a bounded plan, per-call accounting, cancellation/staleness semantics, consent scope and adversarial fixtures.

Do not add a second manual prose workflow, hidden autonomous progression, numerical relationship grinding or compatibility adapters merely to retain old UI vocabulary. Such changes alter the approved game and need a deliberate product decision rather than incidental refactoring.

08 Bounded consistency pipeline

Branch snapshots select quality_mode off, standard, memory or both. A server-owned plan hashes mode, role/provider/model identities, endpoint, timeout and maximum calls into a review identity. Quality replies require that identity. Credentials are never part of the hash or public plan. All required roles are resolved before dispatch and every subsequent boundary; stale story or provider changes abort.

The pipeline proposes a draft, validates its structured effects without mutation, then optionally asks the selected standard and/or memory roles to review it. Reviewers return approval or bounded issues with direct candidate quotations, never canon changes. A single repair uses the original authoritative context. The replacement passes the same structural checks and every selected review. Only the final accepted beat enters the ordinary atomic commit. Model approval does not bypass application-owned adjudication, evidence or character ownership.

Schema 21 adds fiction_calls beneath fiction_requests. Each call has a bounded index, role, purpose, model, status and billing data. Off caps at one call, one reviewer at four, and both at six. Transport retries are disabled. Spend is the union of call rows and legacy parent-only purchases, not their double-counted sum. Mixed known/unknown costs survive failure and restart. Reader APIs expose bounded call metadata, not candidates or review explanations. Saves export aggregate spend and the branch setting, never call/replay identities or consent.

09 People and episode framing

Relationship facts optionally identify a qualitative facet and a directed cast pair. Affection, trust, cooperation and expectations remain independent. The develop effect changes only a relationship's description, with exact passage or input evidence, immutable prior provenance and the inhabited-person boundary. It cannot rewrite world facts, change the facet or grant a challenge.

Episode snapshots hold a public question, up to six public goal identifiers, descriptive phase and a payoff beat. A narrated goal-resolution change can move the episode to payoff; a later scene permits aftermath. No phase changes the player-owned active/ended status. Questions, phases and evidence restore through rewind and copy-import. The director offers rest after a recorded payoff.

The local recap endpoint reads only current ancestry: three narrated summaries, six active public commitments and twelve active public relationships. Kind, visibility and status are filtered before bounded memory selection. No provider, whole-history prompt, autonomous character queue or offline simulation is added.

10 Fourth-wall permission

Living-world snapshots carry a Never/Rarely/Freely preference and the last narrated scene index containing an accepted address. Never is the default; Story-shaping and out-of-story Ask disable character asides. Rarely requires a six-scene index gap; Freely permits consecutive fitting addresses. Preference changes do not reset the cooldown. The same snapshot/save graph preserves it without a separate timer.

The existing narration response may include one bounded structured aside. The server checks permission, cast identity and character ownership, appends the named address to saved prose, and advances its index in the same atomic commit. Effects and challenge evidence are validated against ordinary prose, not appended asides. There is no extra provider request. The protocol cannot prove that unrestricted model prose contains no unstructured fourth-wall language.

11 Clear influence and evidence

Reader directions carry moment or ongoing scope. Only a successful ongoing Steer replaces branch-local focus; one-moment detours and failures do not. Locally derived invitations use public state and fill drafts only. Explicit challenge review is a read-only, revision-checked endpoint: it reveals a prior ruling only when unchanged. Reply commits recheck that same revision, so a free review cannot silently become a newly billable attempt after a concurrent change. A free repeated ruling remains available when a former provider configuration is unavailable.

Public memory search filters before its result bound, and source reads verify active ancestry. Changes can carry an earlier evidence identity; save validation requires it on parent ancestry and import remaps it. The shelf uses bounded 80-story pages instead of silently hiding everything after 200 stories.

12 Memory and adjudication foundations

The bounded snapshot cache is not the only copy of story facts. Immutable initial facts and per-beat changes form a branch-local version history. Retrieval folds the latest version of each fact on the selected ancestry, excludes explicit retirements, ranks relevant entries and returns a bounded result. Working-set compaction drops cached entries only; correction and export preserve their history.

Story-shaping and Living-world are independent of character ownership and severity. Structured challenges define named approaches and explicit evidence requirements. Application code adjudicates from recorded public facts and character knowledge; models receive the decision, not permission to rewrite it. A matching prior basis replays locally without a provider call. The request journal, stale checks and atomic commit still apply. Model-returned outcome/evidence are checked, but this is not a semantic proof that every generated sentence agrees. Open-ended dialogue remains model-dependent and must not be advertised as mechanically adjudicated.

13 Media and portability boundary

The new game owns immutable branch-state illustration placements and story-scoped normalized raster assets. Media never becomes prose or truth. A dedicated Illustrator role uses the game request journal and exact revision/path checks. Upload/description changes remain local. Files are staged under random keys; asset insertion, placement snapshot and terminal paid accounting share one database transaction.

The game projects only the selected ancestry's prose and placements into the existing PublicationDocument. EPUB splits images into individual fixed-layout spine items; prose stays reflowable. All adapters share one privacy-filtered document. Playable saves are separate bounded gzip-JSON graphs carrying every snapshot and image, with no credentials or request authority. Validation uses ancestry intervals to reject future/cross-path evidence before a transactional copy with remapped identities.

The /api/fiction/catalog namespace is independent of the retired catalogue API. Schema 22 adds fiction_templates, fiction_template_assets and image requests; schema 23 adds the separate fiction_template_drafts journal. Entries have revisions, bounded typed fields and one current normalized image. CRUD is local. AI reference development is seed-based, idempotent and capped at two calls only when invalid JSON needs one correction; its result remains unsaved until explicit CRUD. Explicit image generation is single-attempt and accounted. Deletion scrubs content and removes its owned image while retaining both spend journals.

Story creation takes trusted template snapshots and fresh copies of selected images, then inserts the story graph and image ownership in one transaction. Failure discards only its uncommitted copies. There is no live catalogue foreign key in story state. World lore and character motives/background remain private bounded narrator context; public state exposes only visible reference metadata. A Scribe supplies craft, not cast ownership. Catalogue edits cannot rewrite a running story.

Branch-local visuals target cover, world, Scribe or a current cast identity. Historical assets remain available to rewind and saves. Books include the current cover and passage illustrations, not reference portraits or private setup notes.